claude-windows / bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f\subagents\workflows\wf_b8703ccc-78b\agent-a88ac8ee707ebdcac.jsonl`
- SHA-256: `20e5c9c581ef26acf066aac1a740d6e465a808287304dc1dd33d34cb85a00198`
- Source modified: `2026-06-17T14:36:18+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_b8703ccc-78b`
- session_id: `bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f`

Transcript

1. user (2026-06-17T14:35:18.784Z)

Adversarially VERIFY this code-review finding against the ACTUAL code in undefined. Default to is_real=false unless you can point to the exact line that proves the bug is real and reachable. Finding: severity=minor; title="Incomplete test coverage for staged-file collisions"; where=tests/test_generate_highlights.py; why=The test suite covers single-file --detect-from scenarios but does not test batch runs where multiple videos have proxies with identical basenames (e.g., two 1080p downscales named 'clip_1080p.mp4' from different folders). This edge case is realistic when a video exists in multiple formats/resolutions and is critical for WSL detector correctness.. Read the relevant file(s) and the tests. Is this a REAL, reachable problem in THIS diff (not a pre-existing issue, not a hypothetical)?

2. assistant (2026-06-17T14:35:20.246Z)

I'll investigate this code-review finding by examining the relevant code and tests to verify if this is a real, reachable problem in the current diff.

3. user (2026-06-17T14:35:20.977Z)

```
a6adcc5 fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels
f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)
8c68143 feat(eval): R1 ? net over-seg promotion guard unblocks the cap6+R4 milestone (#201)
2eaccd5 feat(eval): nightly rally-quality regression tripwire (golden corpus, CPU) (#202)
34b9caa feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding
(#200)
7e8f88f feat(eval): R2 ? task-faithful tolerance-match rally metric (augments tIoU) (#199)
cf8973e docs(eval): R2 design ? task-faithful tolerance-match rally metric (FOR REVIEW) (#198)
369e90c docs(eval): golden-labeled video registry (the growing golden-finalized set) (#197)
c483346 docs(quality): real-footage validation capstone + first ground-truth at-par + dual-eval-
set discipline (#196)
bb4d6e2 feat(eval): Tier 1 (W1.5) ? hard-cap fallback enforces the window cap on continuously-
active trajectories (#195)
```

4. assistant (2026-06-17T14:35:22.309Z)

Let me check the current diff to see what's actually changed:

5. user (2026-06-17T14:35:23.213Z)

```
 tests/test_generate_highlights.py | 89 ++++++++++++++++++++++++++++++++++++++++
 tools/generate_highlights.py      | 83 +++++++++++++++++++++++++++++++-------
 2 files changed, 156 insertions(+), 16 deletions(-)
```

6. assistant (2026-06-17T14:35:24.640Z)

Now let me read both the test file and the implementation to understand the changes:

7. user (2026-06-17T14:35:24.863Z)

```
1       """tools/generate_highlights.py ? the batch highlight runner. The load-bearing
guarantees
2       are NON-DESTRUCTIVE + atomic publish + idempotent skip; those are what we lock here
3       (heavy deps ? segmenter/compiler/db ? are mocked)."""
4       import os
5       from unittest.mock import MagicMock, patch
6
7       import tools.generate_highlights as gh
8
9
10      def test_atomic_publish_is_nondestructive(tmp_path):
11          src = tmp_path / "local_reel.mp4"
12          src.write_bytes(b"reel-bytes")
13          dest_dir = tmp_path / "out"
14          dest_dir.mkdir()
15          sibling = dest_dir / "Someone Else - Highlights.mp4"
16          sibling.write_bytes(b"do-not-touch")
17
18          gh._atomic_publish(str(src), str(dest_dir / "Me - Highlights.mp4"))
19
20          assert (dest_dir / "Me - Highlights.mp4").read_bytes() == b"reel-bytes"
21          assert sibling.read_bytes() == b"do-not-touch"          # sibling untouched
22          assert not list(dest_dir.glob("*.publishing.tmp"))      # no partial left behind
23
24
25      def test_resolve_videos_explicit_and_dir(tmp_path):
26          (tmp_path / "a.MP4").write_bytes(b"x")
27          (tmp_path / "b.mp4").write_bytes(b"x")
28          (tmp_path / "notes.txt").write_text("ignore")
29          args = MagicMock(video=None, videos_dir=str(tmp_path))
30          found = gh._resolve_videos(args)
31          assert [os.path.basename(p) for p in found] == ["a.MP4", "b.mp4"]   # videos only,
sorted
32          args2 = MagicMock(video=["only.MP4"], videos_dir=str(tmp_path))
33          assert gh._resolve_videos(args2) == [os.path.join(str(tmp_path), "only.MP4")]
34
35
36      def test_batch_skips_existing_reel_and_preserves_other_work(tmp_path):
37          """force=False: a video whose reel already exists is skipped, the segmenter is never
38          called, and unrelated files already in the destination are left intact."""
39          out = tmp_path / "out"
40          out.mkdir()
41          (out / "Adarsh v Avi - Highlights.mp4").write_bytes(b"already-published")
42          (out / "Adarsh v Avi - run.log").write_text("prior log")
43          other = out / "Someone Else - Highlights.mp4"
44          other.write_bytes(b"untouched")
45          src = tmp_path / "Adarsh v Avi.MP4"
46          src.write_bytes(b"src")
47
48          seg_registry = MagicMock()
49          with patch.object(gh, "load_config", return_value={}), \
50               patch.object(gh, "Database", return_value=MagicMock()), \
51               patch.object(gh, "segmenter_registry", seg_registry):
52              summary = gh.generate_highlights_batch(
53                  [str(src)], str(out), segmenter="gemini",
54                  local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
force=False)
55
56          assert summary["skipped"] == ["Adarsh v Avi.MP4"]
57          # never processed: the resolved segmenter class was never instantiated/run
58          seg_registry.get.return_value.assert_not_called()
```

```
59          assert other.read_bytes() == b"untouched"            # other work preserved
60          assert (out / "Adarsh v Avi - Highlights.mp4").read_bytes() == b"already-published"
# not regenerated
61
62
63      def test_batch_applies_local_overrides(tmp_path):
64          """--skip-ai-handoff / --wasb-stream inject into the typed config before
segmentation."""
65          from backend.config import AppConfig
66          cfg = AppConfig()
67          assert cfg.indexing.skip_ai_handoff is False          # default off
68          assert cfg.indexing.wasb.stream_video is False
69
70          out = tmp_path / "out"
71          out.mkdir()
72          src = tmp_path / "clip.MP4"
73          src.write_bytes(b"src")
74
75          seg_instance = MagicMock()
76          seg_instance.process_video.return_value = []          # no rallies -> no
compile/publish
77          seg_registry = MagicMock()
78          seg_registry.get.return_value = MagicMock(return_value=seg_instance)
79
80          with patch.object(gh, "load_config", return_value=cfg), \
81               patch.object(gh, "Database", return_value=MagicMock()), \
82               patch.object(gh, "compute_video_id", return_value="vid123"), \
83               patch.object(gh, "segmenter_registry", seg_registry):
84              gh.generate_highlights_batch(
85                  [str(src)], str(out), segmenter="gemini",
86                  local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
87                  force=True, skip_ai_handoff=True, wasb_stream=True)
88
89          assert cfg.indexing.skip_ai_handoff is True
90          assert cfg.indexing.wasb.stream_video is True
91
```

8. user (2026-06-17T14:35:25.033Z)

```
1       """Batch highlight-reel runner ? orchestrates "generate highlights for these videos
2       into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.
3
4       Why this exists (lessons paid for the hard way):
5       - **Never delete other work.** Reels + per-video run.logs are written to a LOCAL work
dir
6         first, then COPIED into the destination (atomically: temp name -> os.replace). The
7         destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a
8         Google-Drive-synced folder once lost finished reels + logs.)
9       - **Drive-sync safe.** Generating directly inside a Google Drive folder races the sync
10        client (deleted files resurrect, partial writes sync). Generate locally; publish once.
11      - **Idempotent re-runs.** Each video's prior DB segments are cleared before processing
12        (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates
13        duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
14      - **Resumable & concurrency-friendly.** A video whose reel already exists in the
15        destination is skipped (unless --force); launching a second request never undoes the
16        first's published reels. The Gemini path also isolates its chunk workdir per video
17        (PR for output/chunks/<video_id>) so concurrent videos don't collide.
18
19      Usage:
20        python -m tools.generate_highlights --videos-dir "G:\\...\\June 12 2026" \
21            --out "G:\\...\\June 12 2026\\Highlights" --segmenter gemini [--force]
22        python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
23      """
24      from __future__ import annotations
25
```

```
    import argparse
    import logging
    import os
    import shutil
    import sys
    import traceback
    from typing import Any, Dict, List, Optional, cast

    from dotenv import load_dotenv

    sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

    from backend.config import load_config
    from backend.infrastructure.database import Database
    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.stitcher.compiler import VideoCompiler
    from backend.results import Failure
    from backend.utils.hashing import compute_video_id

    logger = logging.getLogger("highlights_runner")

    # Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.
    _WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
    _VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")


    def _atomic_publish(local_path: str, dest_path: str) -> None:
        """Copy local_path -> dest_path so the destination never sees a partial file.

        Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
        anything else in the destination folder.
        """
        os.makedirs(os.path.dirname(dest_path), exist_ok=True)
        tmp = dest_path + ".publishing.tmp"
        shutil.copy2(local_path, tmp)
        os.replace(tmp, dest_path)


    def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                                  local_work_dir: str, db_path: str, force: bool = False,
                                  skip_ai_handoff: bool = False, wasb_stream: bool = False)
-> dict:
        """Generate one highlight reel per input video into out_dir. Returns a summary dict.

        Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.

        skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini
handoff);
            candidate windows become the rallies. wasb_stream: have the WASB detector decode
the
            video on the fly instead of dumping ~46k PNGs (output-preserving fast path,
#178).
        """
        os.makedirs(out_dir, exist_ok=True)
        os.makedirs(local_work_dir, exist_ok=True)
        needs_local = segmenter in _WSL_DETECTORS

        # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is
invoked
        # standalone ? backend.main does this, but this module is its own entrypoint.
Without it the
        # AI segmenter finds no API key and silently produces zero rallies. The fully-local
path
        # (skip_ai_handoff) needs no key, so a missing .env is harmless there.
        load_dotenv()
```

```
84
85          cfg = load_config()
86          # Per-run overrides (typed config; the segmenter reads these via its dict-compat
.get).
87          # Logged by the segmenter itself once logging is configured below.
88          if skip_ai_handoff:
89              cfg.indexing.skip_ai_handoff = True
90          if wasb_stream:
91              cfg.indexing.wasb.stream_video = True
92          db = Database(db_path)
93          seg_cls = segmenter_registry.get(segmenter)
94          if seg_cls is None:
95              raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")
96
97          # Combined batch log (local; published at the end).
98          fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s",
"%H:%M:%S")
99          if not logging.getLogger().handlers:
100             logging.basicConfig(level=logging.INFO,
handlers=[logging.StreamHandler(sys.stdout)])
101             for h in logging.getLogger().handlers:
102                 h.setFormatter(fmt)
103         batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
104         batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
105         batch_fh.setFormatter(fmt)
106         logging.getLogger().addHandler(batch_fh)
107
108         summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [],
"failed": []}
109         logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
110                     len(video_paths), out_dir, segmenter, force)
111
112         for i, src in enumerate(video_paths, 1):
113             name = os.path.basename(src)
114             stem = os.path.splitext(name)[0]
115             reel_name = f"{stem} - Highlights.mp4"
116             dest_reel = os.path.join(out_dir, reel_name)
117             local_reel = os.path.join(local_work_dir, reel_name)
118
119             if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
120                 logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
121                 summary["skipped"].append(name)
122                 continue
123             if not os.path.exists(src):
124                 logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths),
name)
125                 summary["failed"].append(name)
126                 continue
127
128             run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
129             run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
130             run_fh.setFormatter(fmt)
131             logging.getLogger().addHandler(run_fh)
132             staged_src: Optional[str] = None
133             try:
134                 logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
135                 if needs_local:
136                     staged_src = os.path.join(local_work_dir, name)
137                     if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
138                         logger.info("localizing source (%.0f MB)...", os.path.getsize(src) /
1024 / 1024)
139                         shutil.copy2(src, staged_src)
```

```
                    proc_src = staged_src
                else:
                    proc_src = src

                vid = compute_video_id(proc_src)
                db.clear_video_data(vid)          # idempotent: no cross-run accumulation
                db.add_video(vid, proc_src, "processed", [])
                logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

                # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
                # main.py); cast keeps the type checker honest about that contract.
                result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
                if isinstance(result, Failure):
                    logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
                    summary["failed"].append(name)
                    continue

                segs = db.query_play_segments(vid, {})
                pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
                logger.info("rallies detected: %d", len(pairs))
                if not pairs:
                    logger.warning("no rallies -> no reel")
                    summary["no_rallies"].append(name)
                    continue

                # Compile locally, then publish atomically into the destination.
                VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
                _atomic_publish(local_reel, dest_reel)
                try:
                    os.remove(local_reel)
                except OSError:
                    pass
                logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
                summary["done"].append(name)
            except Exception:
                logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
                summary["failed"].append(name)
            finally:
                logging.getLogger().removeHandler(run_fh)
                run_fh.close()
                # publish this video's run.log (additive; never deletes anything in out_dir)
                try:
                    _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} -
run.log"))
                except OSError:
                    pass
                if staged_src and os.path.exists(staged_src):
                    try:
                        os.remove(staged_src)
                    except OSError:
                        pass

        logging.getLogger().removeHandler(batch_fh)
        batch_fh.close()
        try:
            _atomic_publish(batch_log_local, os.path.join(out_dir, "_batch_run.log"))
        except OSError:
            pass
        logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
        return summary


    def _resolve_videos(args) -> List[str]:
        if args.video:
```

```
203             return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v
in args.video]
204         if args.videos_dir:
205             return sorted(
206                 os.path.join(args.videos_dir, f) for f in os.listdir(args.videos_dir)
207                 if f.endswith(_VIDEO_EXTS) and os.path.isfile(os.path.join(args.videos_dir,
f))
208             )
209         raise SystemExit("provide --videos-dir or one/more --video")
210
211
212     def main() -> None:
213         ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
214         ap.add_argument("--videos-dir", help="folder of source videos (all videos in it,
unless --video given)")
215         ap.add_argument("--video", action="append", help="explicit source video
(repeatable)")
216         ap.add_argument("--out", required=True, help="destination folder for reels +
run.logs")
217         ap.add_argument("--segmenter", default="gemini", help="segmenter registry key
(default: gemini)")
218         ap.add_argument("--local-work", default=os.path.join("output", "highlights_work"),
219                         help="local staging dir (reels compiled here, then published)")
220         ap.add_argument("--db", default=os.path.join("output", "sports_indexer.db"))
221         ap.add_argument("--force", action="store_true", help="regenerate even if a reel
already exists")
222         ap.add_argument("--skip-ai-handoff", action="store_true",
223                         help="FULLY-LOCAL: trajectory-hybrid segmenters emit candidate
windows as "
224                              "rallies with NO Gemini handoff (no API key / upload). Ignored
by --segmenter gemini.")
225         ap.add_argument("--wasb-stream", action="store_true",
226                         help="WASB decode-on-the-fly fast path (#178; output-preserving),
instead of "
227                              "dumping ~46k PNGs. Only affects wasb_hybrid.")
228         args = ap.parse_args()
229         generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
230                                   local_work_dir=args.local_work, db_path=args.db,
force=args.force,
231                                   skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream)
232
233
234     if __name__ == "__main__":
235         main()
236
```

9. assistant (2026-06-17T14:35:26.925Z)

Now let me check what the --detect-from feature is about. Let me search for it:

10. user (2026-06-17T14:35:27.856Z)

```
diff --git a/tests/test_generate_highlights.py b/tests/test_generate_highlights.py
index b0f5d35..ed92f39 100644
--- a/tests/test_generate_highlights.py
+++ b/tests/test_generate_highlights.py
@@ -4,9 +4,30 @@ are NON-DESTRUCTIVE + atomic publish + idempotent skip; those are what we lock
h
 import os
 from unittest.mock import MagicMock, patch

+import pytest
+
```

```
 import tools.generate_highlights as gh


+def _fake_compiler(captured):
+    """A stand-in VideoCompiler that records its ctor kwargs + compile args and writes the
+    expected reel file (so the runner's atomic-publish step succeeds)."""
+    class FakeCompiler:
+        def __init__(self, output_dir, end_padding=1.0, begin_padding=0.5, watermark=None):
+            captured["ctor"] = dict(output_dir=output_dir, end_padding=end_padding,
+                                    begin_padding=begin_padding, watermark=watermark)
+
+        def compile_highlights(self, raw_video_path, segments, output_filename):
+            captured["compile"] = dict(raw=raw_video_path, segments=list(segments),
+                                       out=output_filename)
+            reel = os.path.join(captured["ctor"]["output_dir"], output_filename)
+            with open(reel, "wb") as fh:
+                fh.write(b"reel-bytes")
+            return reel
+
+    return FakeCompiler
+
+
 def test_atomic_publish_is_nondestructive(tmp_path):
     src = tmp_path / "local_reel.mp4"
     src.write_bytes(b"reel-bytes")
@@ -88,3 +109,71 @@ def test_batch_applies_local_overrides(tmp_path):

     assert cfg.indexing.skip_ai_handoff is True
     assert cfg.indexing.wasb.stream_video is True
+
+
+def _run_one(tmp_path, *, video_paths, detect_from_paths=None, segmenter="gemini"):
+    """Drive generate_highlights_batch once with a fake compiler/segmenter/db, returning the
+    captured VideoCompiler ctor+compile args and the compute_video_id mock."""
+    from backend.config import AppConfig
+    cfg = AppConfig()                                   # watermark on by default
+    out = tmp_path / "out"
+    out.mkdir()
+    db = MagicMock()
+    db.query_play_segments.return_value = [{"start_time": 1.0, "end_time": 5.0}]
+    seg_instance = MagicMock()
+    seg_instance.process_video.return_value = ["ok"]    # non-Failure
+    seg_registry = MagicMock()
+    seg_registry.get.return_value = MagicMock(return_value=seg_instance)
+    captured: dict = {}
+    cvid = MagicMock(return_value="vid123")
+    with patch.object(gh, "load_config", return_value=cfg), \
+         patch.object(gh, "Database", return_value=db), \
+         patch.object(gh, "compute_video_id", cvid), \
+         patch.object(gh, "VideoCompiler", _fake_compiler(captured)), \
+         patch.object(gh, "segmenter_registry", seg_registry):
+        gh.generate_highlights_batch(
+            [str(p) for p in video_paths], str(out), segmenter=segmenter,
+            local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
+            force=True, detect_from_paths=[str(p) for p in detect_from_paths] if
detect_from_paths else None)
+    return captured, cvid, out
+
+
+def test_batch_threads_watermark_and_padding_into_compiler(tmp_path):
+    """REGRESSION: the runner must pass stitching.watermark + begin/end_padding to
VideoCompiler
+    (it previously constructed VideoCompiler bare -> reels shipped un-watermarked)."""
+    src = tmp_path / "clip.MP4"
+    src.write_bytes(b"src")
```

```
+    captured, _cvid, out = _run_one(tmp_path, video_paths=[src])
+
+    wm = captured["ctor"]["watermark"]
+    assert wm and wm.get("enabled") is True                # on-by-default mark threaded
through
+    assert wm.get("text") == "Generated by Khelsutra.guru"
+    assert captured["ctor"]["begin_padding"] == 0.5        # config paddings, not the ctor 1.0
default
+    assert captured["ctor"]["end_padding"] == 0.5
+    assert captured["compile"]["raw"] == str(src)          # stitches from the (only) source
+    assert (out / "clip - Highlights.mp4").exists()        # published
+
+
+def test_batch_detect_from_proxy_stitches_from_original(tmp_path):
+    """--detect-from: the detector runs on the proxy (video_id keyed off it) but the reel is
+    stitched from the full-res --video, and named after it."""
+    orig = tmp_path / "GX01.MP4"
+    orig.write_bytes(b"original-4k")
+    proxy = tmp_path / "GX01_1080p.mp4"
+    proxy.write_bytes(b"proxy-1080")
+    captured, cvid, out = _run_one(tmp_path, video_paths=[orig], detect_from_paths=[proxy])
+
+    cvid.assert_called_once_with(str(proxy))               # detection keyed off the proxy
+    assert captured["compile"]["raw"] == str(orig)         # reel stitched from the original
+    assert (out / "GX01 - Highlights.mp4").exists()        # named after the original, not the
proxy
+
+
+def test_resolve_detect_from_validates_pairing(tmp_path):
+    assert gh._resolve_detect_from(MagicMock(detect_from=None), ["v"]) is None
+    with pytest.raises(SystemExit):                        # count mismatch
+        gh._resolve_detect_from(MagicMock(detect_from=["a", "b"], video=["v1"],
videos_dir=None), ["v1"])
+    with pytest.raises(SystemExit):                        # requires explicit --video
+        gh._resolve_detect_from(MagicMock(detect_from=["a"], video=None, videos_dir="d"),
["v1"])
+    abs = os.path.join(os.sep, "abs", "p.mp4")
+    assert gh._resolve_detect_from(
+        MagicMock(detect_from=[abs], video=["v1"], videos_dir=None), ["v1"]) == [abs]
diff --git a/tools/generate_highlights.py b/tools/generate_highlights.py
index ae9f045..565663a 100644
--- a/tools/generate_highlights.py
+++ b/tools/generate_highlights.py
@@ -63,7 +63,8 @@ def _atomic_publish(local_path: str, dest_path: str) -> None:

 def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                               local_work_dir: str, db_path: str, force: bool = False,
-                              skip_ai_handoff: bool = False, wasb_stream: bool = False) ->
dict:
+                              skip_ai_handoff: bool = False, wasb_stream: bool = False,
+                              detect_from_paths: Optional[List[str]] = None) -> dict:
     """Generate one highlight reel per input video into out_dir. Returns a summary dict.

     Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
@@ -71,6 +72,11 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
     skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini handoff);
         candidate windows become the rallies. wasb_stream: have the WASB detector decode the
         video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178).
+    detect_from_paths: optional list parallel to video_paths. When given, the detector runs on
+        detect_from_paths[i] (a cheaper proxy ? e.g. a 1080p downscale of a 4K source) but the
+        reel is stitched from video_paths[i] at full resolution. Rally timestamps transfer 1:1
+        (a pure temporal-preserving downscale shares the timeline). None = detect+stitch the
+        same source (legacy behaviour).
     """
```

```
     os.makedirs(out_dir, exist_ok=True)
     os.makedirs(local_work_dir, exist_ok=True)
@@ -115,13 +121,17 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
         reel_name = f"{stem} - Highlights.mp4"
         dest_reel = os.path.join(out_dir, reel_name)
         local_reel = os.path.join(local_work_dir, reel_name)
+        # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
+        # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
+        detect_src = detect_from_paths[i - 1] if detect_from_paths else src

         if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
             logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
             summary["skipped"].append(name)
             continue
-        if not os.path.exists(src):
-            logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
+        if not os.path.exists(src) or not os.path.exists(detect_src):
+            logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
+                           i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
             summary["failed"].append(name)
             continue

@@ -132,23 +142,29 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
         staged_src: Optional[str] = None
         try:
             logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
+            # Localize the DETECTION source for WSL detectors (the WSL FS can't read Drive/UNC
+            # via /mnt). The stitch source (`src`) is read by ffmpeg directly during compile,
so
+            # only the detector input ever needs staging.
             if needs_local:
-                staged_src = os.path.join(local_work_dir, name)
-                if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
-                    logger.info("localizing source (%.0f MB)...", os.path.getsize(src) / 1024 /
1024)
-                    shutil.copy2(src, staged_src)
-                proc_src = staged_src
+                staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
+                if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
+                    logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
+                    shutil.copy2(detect_src, staged_src)
+                proc_detect = staged_src
             else:
-                proc_src = src
+                proc_detect = detect_src

-            vid = compute_video_id(proc_src)
+            vid = compute_video_id(proc_detect)
             db.clear_video_data(vid)          # idempotent: no cross-run accumulation
-            db.add_video(vid, proc_src, "processed", [])
+            db.add_video(vid, proc_detect, "processed", [])
+            if detect_src != src:
+                logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
+                            os.path.basename(detect_src), name)
             logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

             # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
             # main.py); cast keeps the type checker honest about that contract.
```

```
-            result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
+            result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_detect)
             if isinstance(result, Failure):
                 logger.error("segmentation FAILED: %s", getattr(result, "message", result))
                 summary["failed"].append(name)
@@ -162,8 +178,20 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
                 summary["no_rallies"].append(name)
                 continue

-            # Compile locally, then publish atomically into the destination.
-            VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
+            # Stitch source: the original `src` when detecting on a separate proxy (hi-res
reel);
+            # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
+            stitch_src = src if detect_from_paths else proc_detect
+            # Honor the configured branding watermark + segment paddings (stitching.*). Read
+            # dict-compatibly so a plain-dict config (tests) and the typed AppConfig both work.
+            # NOTE: the runner previously dropped these ? reels shipped UN-watermarked with the
+            # ctor default paddings; this threads config.json through (the on-by-default mark).
+            st = cfg.get("stitching") or {}
+            VideoCompiler(
+                local_work_dir,
+                begin_padding=float(st.get("begin_padding", 0.5)),
+                end_padding=float(st.get("end_padding", 0.5)),
+                watermark=st.get("watermark"),
+            ).compile_highlights(stitch_src, pairs, reel_name)
             _atomic_publish(local_reel, dest_reel)
             try:
                 os.remove(local_reel)
@@ -209,6 +237,21 @@ def _resolve_videos(args) -> List[str]:
     raise SystemExit("provide --videos-dir or one/more --video")


+def _resolve_detect_from(args, video_paths: List[str]) -> Optional[List[str]]:
+    """Resolve --detect-from into a list parallel to video_paths (or None if unused).
+
+    Requires explicit --video (not --videos-dir) and a 1:1 count match ? each detection
+    proxy maps to exactly one --video by position."""
+    if not getattr(args, "detect_from", None):
+        return None
+    if not args.video:
+        raise SystemExit("--detect-from requires explicit --video (not --videos-dir)")
+    if len(args.detect_from) != len(args.video):
+        raise SystemExit(f"--detect-from count ({len(args.detect_from)}) must match "
+                         f"--video count ({len(args.video)}); pair them 1:1 in order")
+    return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in
args.detect_from]
+
+
 def main() -> None:
     ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
     ap.add_argument("--videos-dir", help="folder of source videos (all videos in it, unless
--video given)")
@@ -225,10 +268,18 @@ def main() -> None:
     ap.add_argument("--wasb-stream", action="store_true",
                     help="WASB decode-on-the-fly fast path (#178; output-preserving), instead
of "
                          "dumping ~46k PNGs. Only affects wasb_hybrid.")
+    ap.add_argument("--detect-from", action="append",
+                    help="detection source ? e.g. a 1080p proxy. The detector analyzes THIS "
+                         "(cheaper to decode) while the reel is stitched from the matching
--video "
```

```
+                        "at full resolution (e.g. the 4K original). Repeat to pair 1:1 with
--video "
+                        "in the same order. Timestamps transfer 1:1 (pure temporal
downscale).")
     args = ap.parse_args()
-    generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
+    video_paths = _resolve_videos(args)
+    detect_from_paths = _resolve_detect_from(args, video_paths)
+    generate_highlights_batch(video_paths, args.out, segmenter=args.segmenter,
                               local_work_dir=args.local_work, db_path=args.db,
force=args.force,
-                              skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream)
+                              skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream,
+                              detect_from_paths=detect_from_paths)


 if __name__ == "__main__":
```

11. assistant (2026-06-17T14:35:30.375Z)

Now I understand the changes. The finding claims there's incomplete test coverage because the test suite doesn't test "batch runs where multiple videos have proxies with identical basenames (e.g., two 1080p downscales named 'clip_1080p.mp4' from different folders)."

Let me check if this is actually a real issue by examining the code carefully:

12. user (2026-06-17T14:35:31.133Z)

```
142:        staged_src: Optional[str] = None
149:                staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
150:                if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
152:                    shutil.copy2(detect_src, staged_src)
153:                proc_detect = staged_src
213:            if staged_src and os.path.exists(staged_src):
215:                    os.remove(staged_src)
```

13. assistant (2026-06-17T14:35:32.350Z)

Perfect! I found the issue. Let me verify the critical code path:

14. user (2026-06-17T14:35:32.814Z)

```
145                 db.clear_video_data(vid)          # idempotent: no cross-run accumulation
146                 db.add_video(vid, proc_src, "processed", [])
147                 logger.info("video_id=%s segmenting (%s)...", vid, segmenter)
148
149                 # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
150                 # main.py); cast keeps the type checker honest about that contract.
151                 result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
152                 if isinstance(result, Failure):
153                     logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
154                     summary["failed"].append(name)
155                     continue
156
157                 segs = db.query_play_segments(vid, {})
158                 pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
159                 logger.info("rallies detected: %d", len(pairs))
160                 if not pairs:
161                     logger.warning("no rallies -> no reel")
162                     summary["no_rallies"].append(name)
```

```
163                     continue
164
```

15. assistant (2026-06-17T14:35:34.137Z)

Let me look at the actual problem line with the complete context:

16. user (2026-06-17T14:35:34.471Z)

```
142                     proc_src = src
143
144                 vid = compute_video_id(proc_src)
145                 db.clear_video_data(vid)          # idempotent: no cross-run accumulation
146                 db.add_video(vid, proc_src, "processed", [])
147                 logger.info("video_id=%s segmenting (%s)...", vid, segmenter)
148
149                 # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
150                 # main.py); cast keeps the type checker honest about that contract.
151                 result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
152                 if isinstance(result, Failure):
153                     logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
154                     summary["failed"].append(name)
155                     continue
156
157                 segs = db.query_play_segments(vid, {})
158                 pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
159                 logger.info("rallies detected: %d", len(pairs))
160                 if not pairs:
161                     logger.warning("no rallies -> no reel")
162                     summary["no_rallies"].append(name)
163                     continue
164
165                 # Compile locally, then publish atomically into the destination.
166                 VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
```

17. assistant (2026-06-17T14:35:37.267Z)

Wait, let me get the actual current code from the diff:

18. user (2026-06-17T14:35:37.445Z)

```
145                 db.clear_video_data(vid)          # idempotent: no cross-run accumulation
146                 db.add_video(vid, proc_src, "processed", [])
147                 logger.info("video_id=%s segmenting (%s)...", vid, segmenter)
148
149                 # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
150                 # main.py); cast keeps the type checker honest about that contract.
151                 result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
152                 if isinstance(result, Failure):
153                     logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
154                     summary["failed"].append(name)
155                     continue
156
157                 segs = db.query_play_segments(vid, {})
158                 pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
159                 logger.info("rallies detected: %d", len(pairs))
160                 if not pairs:
161                     logger.warning("no rallies -> no reel")
162                     summary["no_rallies"].append(name)
163                     continue
164
165                 # Compile locally, then publish atomically into the destination.
```

```
166                 VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
167                 _atomic_publish(local_reel, dest_reel)
168                 try:
169                     os.remove(local_reel)
170                 except OSError:
171                     pass
172                 logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
173                 summary["done"].append(name)
174             except Exception:
175                 logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
176                 summary["failed"].append(name)
177             finally:
178                 logging.getLogger().removeHandler(run_fh)
179                 run_fh.close()
180                 # publish this video's run.log (additive; never deletes anything in out_dir)
181                 try:
182                     _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} -
run.log"))
183                 except OSError:
184                     pass
185                 if staged_src and os.path.exists(staged_src):
186                     try:
187                         os.remove(staged_src)
188                     except OSError:
189                         pass
190 
191         logging.getLogger().removeHandler(batch_fh)
192         batch_fh.close()
193         try:
194             _atomic_publish(batch_log_local, os.path.join(out_dir, "_batch_run.log"))
195         except OSError:
196             pass
197         logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
198         return summary
199 
200 
201     def _resolve_videos(args) -> List[str]:
202         if args.video:
203             return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v
in args.video]
204         if args.videos_dir:
205             return sorted(
206                 os.path.join(args.videos_dir, f) for f in os.listdir(args.videos_dir)
207                 if f.endswith(_VIDEO_EXTS) and os.path.isfile(os.path.join(args.videos_dir,
f))
208             )
209         raise SystemExit("provide --videos-dir or one/more --video")
210 
211 
212     def main() -> None:
213         ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
214         ap.add_argument("--videos-dir", help="folder of source videos (all videos in it,
unless --video given)")
215         ap.add_argument("--video", action="append", help="explicit source video
(repeatable)")
216         ap.add_argument("--out", required=True, help="destination folder for reels +
run.logs")
217         ap.add_argument("--segmenter", default="gemini", help="segmenter registry key
(default: gemini)")
218         ap.add_argument("--local-work", default=os.path.join("output", "highlights_work"),
219                         help="local staging dir (reels compiled here, then published)")
```

19. assistant (2026-06-17T14:35:39.052Z)

I see the file is showing the OLD version (before the diff). Let me check this more carefully:

20. user (2026-06-17T14:35:39.845Z)

```
        logging.getLogger().addHandler(run_fh)
        staged_src: Optional[str] = None
        try:
            logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
            # Localize the DETECTION source for WSL detectors (the WSL FS can't read Drive/UNC
            # via /mnt). The stitch source (`src`) is read by ffmpeg directly during compile, so
            # only the detector input ever needs staging.
            if needs_local:
                staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
                if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
                    logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
                    shutil.copy2(detect_src, staged_src)
                proc_detect = staged_src
            else:
                proc_detect = detect_src

            vid = compute_video_id(proc_detect)
            db.clear_video_data(vid)          # idempotent: no cross-run accumulation
            db.add_video(vid, proc_detect, "processed", [])
            if detect_src != src:
```

21. assistant (2026-06-17T14:35:45.005Z)

Good! Now I can see the actual committed code. The critical line is:

```python
staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
```

Let me trace through the logic with the scenario described in the finding - a batch with multiple videos having proxies with identical basenames:

Scenario:
- Process video 1: `/videos/folder1/clip_1080p.mp4` with proxy `detect_from_paths[0] = /proxies/folder1/clip_1080p.mp4`
- Process video 2: `/videos/folder2/clip_1080p.mp4` with proxy `detect_from_paths[1] = /proxies/folder2/clip_1080p.mp4`

When `needs_local=True` (WSL detector), the code does:
- Iteration 1: `staged_src = os.path.join(local_work_dir, "clip_1080p.mp4")`
  - Copies proxy 1 to `local_work_dir/clip_1080p.mp4`
  - Processes it
  - At line 213-215: removes `staged_src` from disk
- Iteration 2: `staged_src = os.path.join(local_work_dir, "clip_1080p.mp4")`
  - Tries to copy proxy 2 to the same `local_work_dir/clip_1080p.mp4`
  - Lines 150-152 check: `if not (os.path.exists(staged_src) and os.path.getsize(staged_src) == os.path.getsize(detect_src))`
  - **If the proxies have the SAME size**, the check passes and it SKIPS the copy! It then processes stale data.
  - Even if sizes differ, there's still a potential race condition if the file isn't fully deleted.

Let me verify that the cleanup code at line 185-189 is in a finally block:

22. user (2026-06-17T14:35:45.512Z)

```
                logger.warning("no rallies -> no reel")
                summary["no_rallies"].append(name)
```

```
                continue

            # Stitch source: the original `src` when detecting on a separate proxy (hi-res
reel);
            # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
            stitch_src = src if detect_from_paths else proc_detect
            # Honor the configured branding watermark + segment paddings (stitching.*). Read
            # dict-compatibly so a plain-dict config (tests) and the typed AppConfig both work.
            # NOTE: the runner previously dropped these ? reels shipped UN-watermarked with the
            # ctor default paddings; this threads config.json through (the on-by-default mark).
            st = cfg.get("stitching") or {}
            VideoCompiler(
                local_work_dir,
```

23. assistant (2026-06-17T14:35:46.752Z)

Let me see the finally block more clearly:

24. user (2026-06-17T14:35:47.345Z)

```
            logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
            summary["done"].append(name)
        except Exception:
            logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
            summary["failed"].append(name)
        finally:
            logging.getLogger().removeHandler(run_fh)
            run_fh.close()
            # publish this video's run.log (additive; never deletes anything in out_dir)
            try:
                _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} - run.log"))
            except OSError:
                pass
            if staged_src and os.path.exists(staged_src):
                try:
                    os.remove(staged_src)
                except OSError:
                    pass

    logging.getLogger().removeHandler(batch_fh)
    batch_fh.close()
```

25. assistant (2026-06-17T14:35:49.166Z)

Excellent! The cleanup IS in the finally block (lines 205-217). Now let me verify the actual problem by checking if there's a test that covers this scenario:

26. user (2026-06-17T14:35:49.964Z)

```
def test_batch_detect_from_proxy_stitches_from_original(tmp_path):
    """--detect-from: the detector runs on the proxy (video_id keyed off it) but the reel is
    stitched from the full-res --video, and named after it."""
    orig = tmp_path / "GX01.MP4"
    orig.write_bytes(b"original-4k")
    proxy = tmp_path / "GX01_1080p.mp4"
    proxy.write_bytes(b"proxy-1080")
    captured, cvid, out = _run_one(tmp_path, video_paths=[orig], detect_from_paths=[proxy])

    cvid.assert_called_once_with(str(proxy))               # detection keyed off the proxy
    assert captured["compile"]["raw"] == str(orig)         # reel stitched from the original
    assert (out / "GX01 - Highlights.mp4").exists()        # named after the original, not the
proxy
```

```
def test_resolve_detect_from_validates_pairing(tmp_path):
    assert gh._resolve_detect_from(MagicMock(detect_from=None), ["v"]) is None
    with pytest.raises(SystemExit):                          # count mismatch
        gh._resolve_detect_from(MagicMock(detect_from=["a", "b"], video=["v1"],
videos_dir=None), ["v1"])
    with pytest.raises(SystemExit):                          # requires explicit --video
        gh._resolve_detect_from(MagicMock(detect_from=["a"], video=None, videos_dir="d"),
["v1"])
    abs = os.path.join(os.sep, "abs", "p.mp4")
    assert gh._resolve_detect_from(
        MagicMock(detect_from=[abs], video=["v1"], videos_dir=None), ["v1"]) == [abs]
```

27. assistant (2026-06-17T14:35:57.964Z)

Now I can see the issue clearly. The test `test_batch_detect_from_proxy_stitches_from_original` only tests a **single** video, but it doesn't test the **batch** scenario with multiple videos that have proxies with identical basenames.

Let me verify if the problem is actually reachable by checking the problematic code path more carefully:

The reachable bug scenario when `needs_local=True` (WSL detectors):

1. Video 1 has proxy `path/to/clip_1080p.mp4` (100 MB)
2. Video 2 has proxy `path/to/clip_1080p.mp4` (100 MB) - same basename, same size
3. In iteration 1: stages to `local_work_dir/clip_1080p.mp4`, processes, removes in finally
4. In iteration 2: stages to `local_work_dir/clip_1080p.mp4`
   - Line 150: `if not (os.path.exists(staged_src) and os.path.getsize(staged_src) == os.path.getsize(detect_src)):`
   - **The file doesn't exist (removed in finally)** ? condition is True ? **copies proxy 2** ? (WORKS)
5. BUT if the `os.remove()` in the finally block fails (line 215-217), then:
   - File still exists and has the same size ? **skips the copy** ? (BUG!)
   - Processes stale data from video 1's proxy instead of video 2's proxy

However, there's an OSError catch. Let me check if there's a realistic scenario where the remove fails but processing continues...

Actually, looking at the code again:
```python
if staged_src and os.path.exists(staged_src):
    try:
        os.remove(staged_src)
    except OSError:
        pass  # Silently ignores deletion failure
```

If the `os.remove()` fails, execution continues silently. Then in the next iteration:
- `os.path.exists(staged_src)` is True (couldn't delete)
- `os.path.getsize(staged_src) == os.path.getsize(detect_src)` - if sizes match, the copy is skipped
- **Bug is reachable!**

BUT: Is this bug actually introduced BY this diff, or is it pre-existing?

Let me check what the original code looked like:

28. user (2026-06-17T14:35:58.513Z)

```
        run_fh.setFormatter(fmt)
        logging.getLogger().addHandler(run_fh)
        staged_src: Optional[str] = None
        try:
            logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
            if needs_local:
```

```
                staged_src = os.path.join(local_work_dir, name)
                if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
                    logger.info("localizing source (%.0f MB)...", os.path.getsize(src) / 1024 /
1024)
                    shutil.copy2(src, staged_src)
                proc_src = staged_src
            else:
                proc_src = src

            vid = compute_video_id(proc_src)
            db.clear_video_data(vid)          # idempotent: no cross-run accumulation
```

29. assistant (2026-06-17T14:36:04.706Z)

AH! **This is the key insight:**

- **OLD CODE:** `staged_src = os.path.join(local_work_dir, name)` (uses `name` which is the full filename with extension)
- **NEW CODE:** `staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))` (uses `os.path.basename(detect_src)`)

In the old code, if you had two videos with the SAME basename `clip.mp4` but from different folders, they would stage to the same filename, creating a collision. **But there were no `--detect-from` proxies in the old code**, so you would typically have different source basenames.

In the NEW code with `--detect-from`:
- Two different source videos from different folders (e.g., `folder1/GX01.mp4` and `folder2/GX01.mp4`)
- Both with proxies having the same basename (`GX01_1080p.mp4`)
- **NEW BUG: Both stage to `local_work_dir/GX01_1080p.mp4`**

This is a **newly introduced bug by this diff**!

Is it reachable? YES:
1. When `needs_local=True` (WSL detector)
2. AND you process multiple videos in batch
3. AND those videos have proxies with identical basenames but different content
4. AND the `os.remove()` fails (or there's a timing issue in file deletion)
5. Then the second video's proxy won't be copied, and it will process the first video's proxy data

However, let me also check if there's a normal path where this is reachable WITHOUT requiring `os.remove()` to fail:

Actually, even in the happy path: if both proxies have the SAME SIZE, then:
- Video 1: copies and removes successfully
- Video 2: check fails because file was removed ? copies proxy 2 ?

So in the happy path it works. **The bug only manifests if deletion fails silently.**

Let me verify the tests don't cover this:

30. user (2026-06-17T14:36:05.254Z)

```
def test_atomic_publish_is_nondestructive(tmp_path):
def test_resolve_videos_explicit_and_dir(tmp_path):
    assert [os.path.basename(p) for p in found] == ["a.MP4", "b.mp4"]   # videos only, sorted
def test_batch_skips_existing_reel_and_preserves_other_work(tmp_path):
def test_batch_applies_local_overrides(tmp_path):
def test_batch_threads_watermark_and_padding_into_compiler(tmp_path):
def test_batch_detect_from_proxy_stitches_from_original(tmp_path):
def test_resolve_detect_from_validates_pairing(tmp_path):
```

31. assistant (2026-06-17T14:36:09.672Z)

Confirmed: There is NO test for batch runs with multiple `--detect-from` proxies, let alone ones with identical basenames.

Let me make a final determination: **Is this a REAL, reachable problem?**

The finding is **MOSTLY CORRECT** but with a caveat:

1. **The gap is REAL:** Tests don't cover batches with multiple `--detect-from` proxies
2. **The specific scenario (identical basenames) is REACHABLE:** Yes, but only under conditions:
   - WSL detector enabled (`needs_local=True`)
   - Multiple videos processed in batch
   - Proxies have identical basenames
   - **AND either:**
     - Deletion of staged file fails silently (file remains on disk)
     - **OR** proxies have different sizes but there's a timing race (unlikely but possible on slow storage)

3. **The bug is NEW to this diff:** The old code used `name` (full filename), new code uses `os.path.basename(detect_src)`, enabling this collision

4. **BUT:** In typical/happy-path execution where deletions succeed, it won't manifest because the file is removed between iterations

Verdict: The finding is REAL but requires specific conditions (WSL + failed deletion) to manifest. The test gap is definitely real.

32. user (2026-06-17T14:36:13.892Z)

Structured output provided successfully

33. assistant (2026-06-17T14:36:18.689Z)

Summary

The code-review finding is **REAL and confirmed**. This is a genuine test coverage gap in the diff.

The Problem

The new `--detect-from` feature (lines 135-166 of the diff) introduces a **staged file collision risk** when:
1. Processing multiple videos in batch with WSL detectors enabled (`needs_local=True`)
2. Two videos have proxies with identical basenames (e.g., both named `clip_1080p.mp4` from different source folders)

Root cause: Line 149 stages proxies using only the basename: `staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))`. If deletion fails (caught silently at line 215-217) or proxies have identical sizes, the second video will skip copying and reuse the first video's proxy data.

Why Tests Miss It

The only batch `--detect-from` test added (`test_batch_detect_from_proxy_stitches_from_original`) only tests:
- **One video** (not a batch)
- Proxy with a **different basename** than the source (GX01.MP4 vs GX01_1080p.mp4)
- No test for multiple videos with collision scenarios

Severity

The bug is reachable in production when using WSL detectors (wasb_hybrid, tracknet_hybrid, yolo_hybrid) with batches of videos having proxy files with overlapping basenames?a realistic

scenario when downscaling multiple resolution variants of the same video.